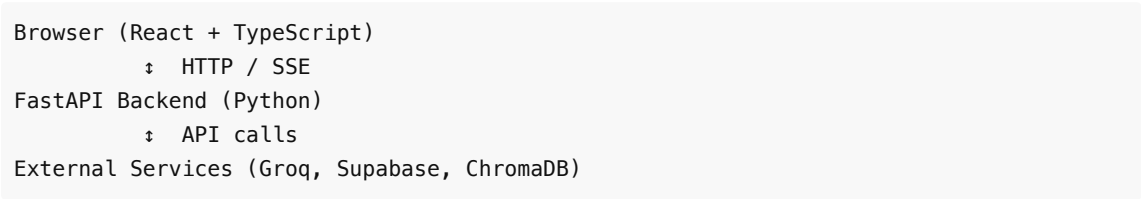


HireReady — Study Guide for Interviews

Full-stack AI resume copilot. Upload a resume, get an ATS score against a job description, see matched/missing skills, then practice tailored interview questions with AI coaching.

Architecture Overview

Three distinct layers:



The core user flow:

1. Upload a resume → backend parses it, splits into chunks, embeds into ChromaDB (local vector DB)
2. Paste a job description → backend compares resume chunks against JD via Groq LLM, returns ATS score + skill gaps
3. Prep for the interview → backend generates tailored questions from resume + JD, gives coaching feedback
4. Chat with the bot → floating chatbot that has your resume as context via ChromaDB RAG

Three external services do all the heavy lifting:

- **Groq** — runs LLMs (text generation) and Whisper (speech-to-text). Fast custom LPU hardware, cheap.
- **Supabase** — two separate jobs: (1) auth (who you are via JWT), (2) storing resume files + analysis history
- **ChromaDB** — a local vector database on the server's disk. Stores resume chunks as searchable vectors so the chatbot and interview prep can find relevant content fast without re-reading the whole document

The core architectural principle: Each external dependency is isolated to exactly one service file. Swap ChromaDB for Pinecone → only `embedder_service.py` changes. Swap Groq for OpenAI → only `llm_service.py` and `interview_service.py` change. Swap Supabase Storage for S3 → only `resume_service.py` changes. Route handlers never know which provider is running.

Tech Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	UI, routing, state management
Build tool	Vite	Dev server + bundler
Routing	React Router v7	Client-side navigation (Home, Interview, Profile)
State	React Context	Persist resume/analysis state across pages
HTTP client	axios	Frontend → backend requests

Auth	Supabase	User accounts, JWT sessions, hosted PostgreSQL + file storage
Backend	FastAPI (Python)	API server, business logic
Validation	Pydantic	Request/response type enforcement
PDF parser	pdfplumber	Extracts text from PDF files
DOCX parser	python-docx	Extracts text from Word files
Embeddings	sentence-transformers	Local model — converts text → vectors
Vector store	ChromaDB	Persists and searches resume chunk embeddings
LLM	Groq (llama-3.3-70b)	ATS scoring, question generation, feedback, chatbot streaming
Transcription	Groq (whisper-large-v3)	Voice answer recording → text in interview prep

Folder Structure

```

backend/app/
  api/
    routes.py          ← Wires all sub-routers together
    resume.py          ← /upload, /analyze, /history (GET+DELETE),
                        /history/{id}/score endpoints
    interview.py        ← /questions, /feedback, /transcribe endpoints
    chat.py            ← /chat SSE streaming endpoint
    github_auth.py      ← GitHub OAuth connect/callback (origin-aware HMAC state)
  services/
    resume_service.py  ← Orchestrates: validate → upload to Supabase Storage →
                        parse via BytesIO → chunk → embed
    parser_service.py   ← Extracts text from PDF/DOCX via BytesIO (no disk writes)
    chunker_service.py  ← Splits text into overlapping 200-word windows
    embedder_service.py ← Embeds chunks + stores/queries ChromaDB; delete_chunks()
    llm_service.py      ← Groq: ATS scoring + is_valid_job_description() (shared
                        validator)
    interview_service.py ← Groq: question generation, feedback, Whisper
  transcription
    chat_service.py     ← Groq: streaming chatbot with resume RAG context
    history_service.py  ← Supabase history: save (INSERT), fetch (grouped), delete,
                        delete_analysis_entry
    jd_fetcher_service.py ← Fetches JD from URL: direct HTTP first, Jina Reader
                        fallback
    github_service.py   ← GitHub API calls (profile, repos)
  models/
    resume.py           ← Pydantic models for upload/analyze/history (includes
                        qualification_gaps)
    interview.py        ← Pydantic models for questions/feedback
    chat.py             ← Pydantic models: ChatMessage, ChatRequest

```

```

frontend/src/
  context/
    ResumeContext.tsx      ← Global state: parseResult, analyzeResult, jd,
qualification_gaps
  AuthContext.tsx          ← Auth state: user, session, signIn, signUp, signOut
  lib/
    supabase.ts            ← Supabase client singleton (reads from .env)
  components/
    ResumeUpload.tsx       ← Upload widget + JD input + ATS results + qualification
gaps
  AuthGate.tsx             ← Sign-in/sign-up page shown at route /
  HowItWorks.tsx           ← Interactive 4-step tutorial section
  Logo.tsx                 ← SVG logo component
  ChatBot.tsx              ← Floating chat widget (SSE streaming, RAG resume context)
  pages/
    InterviewPage.tsx      ← Behavioral + role-specific prep + VoiceMicButton
    ProfilePage.tsx        ← Account info + sign out (sign-in form if guest)
    HistoryPage.tsx        ← Resume history: accordion by file, nested analyses, per-
score delete
  App.tsx                  ← Router, ResumeProvider keyed on user ID, nav, score
guide, ChatBot

backend/app/core/
  config.py                ← Env vars (SUPABASE_URL, SUPABASE_ANON_KEY,
GITHUB_CLIENT_*)
  supabase.py              ← Supabase client singleton
  auth.py                  ← FastAPI dependencies: get_current_user, require_user

```

The key rule: route handlers are thin — they receive the request, call a service, return the response. All logic lives in services. Swap S3 for local disk → change only `resume_service.py`. Swap Pinecone for ChromaDB → change only `embedder_service.py`. Swap OpenAI for Groq → change only `llm_service.py`.

Frontend Architecture: Pages vs Components vs Context

Think of it in terms of **scope**:

Pages — "what am I looking at right now?"

A page is the full screen for a given URL. When you navigate to `/interview`, React renders `InterviewPage.tsx`. When you're at `/history`, it renders `HistoryPage.tsx`. Pages are **associated with a URL** — one per route. They own the layout of everything visible on that screen and orchestrate smaller pieces.

Important: pages do not own routing logic. They are passive — they don't know what URL they're at and don't control navigation. All routing logic lives in `App.tsx`:

```

<Routes>
  <Route path="/home"      element={<HomePage />} />
  <Route path="/interview" element={<InterviewPage />} />

```

```
<Route path="/history" element={<HistoryPage />} />
</Routes>
```

`App.tsx` says "when the URL is `/interview`, render `InterviewPage`." `InterviewPage` has no idea it's associated with `/interview` — it's just a component React Router happens to mount there.

Components — "a reusable piece of UI"

A component is a self-contained chunk of UI that doesn't care where it lives. `ChatBot.tsx` floats on every page — it's not tied to any one route. `ResumeUpload.tsx` is only mounted on the home route but is still a component because it's self-contained and doesn't own any routing logic.

The distinction: **pages compose components**. A page says "put the upload widget here, put the score guide below it." The component just knows how to render itself.

What "mounting" and "unmounting" mean

Mounting = React creates the component and puts it on screen. Think of it like opening an app on your phone — it starts up, allocates memory, displays itself.

Unmounting = React removes the component from the screen and destroys it entirely. Like closing that app — it's gone from memory.

Any `useState` variable lives **inside** the component. The moment the component unmounts, those variables are destroyed. This is why navigation loses state:

```
User is on /home → fileId = "abc-123" (lives inside HomePage)
User navigates to /interview
  → React unmounts HomePage ← fileId destroyed
  → React mounts InterviewPage ← starts fresh, knows nothing
```

Context — "state that needs to survive navigation"

Context is a **global store that lives outside any individual page**. Both `AuthContext` and `ResumeContext` wrap the entire app in `App.tsx`, so their state persists across every navigation.

```
<ResumeContext>          ← never unmounts, always alive
  <Routes>
    <Route path="/home"    element={<HomePage />} />
    <Route path="/interview" element={<InterviewPage />} />
  </Routes>
</ResumeContext>
```

Pages come and go. The context wrapping them never does. `file_id` stored in context survives navigation — it's not inside any page, it's above all of them.

The analogy: writing a note on a sticky note inside a book (gone when you close the book) vs writing it on the wall above the bookshelf (always there no matter which book you open).

```
App.tsx (mounts AuthContext + ResumeContext)
├─ /          → ResumeUpload ← reads/writes ResumeContext
├─ /interview → InterviewPage ← reads file_id + JD from ResumeContext
└─ /history  → HistoryPage
```

```
└─ /profile → ProfilePage
   ChatBot   ← reads from both ResumeContext + AuthContext
```

`ResumeContext` is keyed on `user?.id ?? 'logged-out'` in `App.tsx`. When you log out and a different user logs in, React sees a new key and remounts the provider with fresh empty state — the next user can never see the previous user's resume data.

Backend Architecture: Routes vs Services

The analogy: Routes are waiters, services are kitchen stations.

- **Routes (waiters)** take your order (the HTTP request), hand it to the kitchen, and bring back the food (the response). They don't cook anything.
- **Services (kitchen stations)** each know how to do one thing well.

A route handler looks like this:

```
@router.post("/analyze")
async def analyze_resume(request: AnalyzeRequest, user=Depends(get_current_user)):
    result = await llm_service.analyze(request.file_id, request.job_description)
    await history_service.save(user.id, result)
    return result
```

It's just: receive → call service → return. No Groq calls, no SQL, no business logic.

FastAPI's dependency injection powers `get_current_user`. Adding `user=Depends(get_current_user)` to a route makes FastAPI automatically run that function before the handler. If the JWT is invalid, it returns a 401 and your handler never runs. It's a clean way to enforce auth without repeating the check in every route.

Why this separation matters:

- You can test each service independently without running a full HTTP server
 - You can swap any provider (LLM, vector DB, file storage) by changing exactly one file
 - Routes stay readable — you can understand the API surface without knowing implementation details
-

Pydantic Models

Pydantic is a Python library that lets you define the **shape of data** as a class and automatically validates that incoming data matches that shape before your code ever runs.

Without Pydantic you'd write validation manually:

```
if "file_id" not in request:
    raise ValueError("missing file_id")
if not isinstance(request["job_description"], str):
    raise ValueError("wrong type")
```

With Pydantic you just define:

```
class AnalyzeRequest(BaseModel):
    file_id: str
    job_description: str
```

FastAPI reads the type hint on the route handler (`body: AnalyzeRequest`), automatically parses the incoming JSON into that model, and rejects malformed requests with a 422 before your function runs. You write zero parsing or validation code.

The three model files

models/resume.py — shapes for the upload/analyze/history feature:

- What an `/upload` response looks like (`{ file_id, filename, word_count, sections }`)
- What an `/analyze` request needs (`{ file_id, job_description }`)
- What an `/analyze` response looks like (`{ score, matched_skills, missing_skills, qualification_gaps, summary }`)
- What a history entry looks like (`AnalysisEntry` , `ResumeFile`)

models/interview.py — shapes for interview prep:

- What `/questions` needs (`{ file_id?, job_description }`)
- What `/feedback` needs (`{ question, user_answer, file_id? }`)
- What a question looks like in the response (`{ question, category, hint }`)

models/chat.py — shapes for the chatbot:

- `ChatMessage` — one message (`{ role: "user"|"assistant", content: str }`)
- `ChatRequest` — what `/chat` receives (`{ messages: ChatMessage[], file_id?, job_description? }`)

They're separated by feature area so that adding a field to the analyze response only touches `models/resume.py` — no other model file is affected.

The `/analyze` Endpoint — Line by Line

This is the most important endpoint in the app. Here's every part:

The decorator

```
@router.post("/analyze", response_model=AnalyzeResponse)
```

Registers this function as the handler for `POST /api/resume/analyze` . `router` is mounted in `routes.py` under `/api/resume` , so the full URL is `/api/resume/analyze` . `response_model=AnalyzeResponse` tells FastAPI to validate the return value matches `AnalyzeResponse` before serializing to JSON.

The function signature

```
async def analyze(body: AnalyzeRequest, user: Optional[dict] =
    Depends(get_current_user)):
```

- **async def** — non-blocking. While waiting on Groq or ChromaDB, the server handles other requests instead of freezing.
- **body: AnalyzeRequest** — FastAPI sees this type hint and automatically parses the incoming JSON into the Pydantic model. Missing fields → 422 before the function runs.
- **Depends(get_current_user)** — FastAPI runs `get_current_user` first, which extracts the JWT from the Authorization header and verifies it with Supabase. The result is passed in as `user`. Invalid token → 401, function never runs. `Optional` means guests (no token) still get through — they just won't have history saved.

Gate 1 — word count

```
jd_words = body.job_description.strip().split()
if len(jd_words) < 20:
    raise HTTPException(status_code=422, detail="Job description is too short...")
```

Cheap check, no external calls. Fewer than 20 words → immediate 422.

Gate 2 — semantic validation

```
if not is_valid_job_description(body.job_description):
    raise HTTPException(status_code=422, detail="This doesn't look like a real job description...")
```

Calls `llm_service.is_valid_job_description()` — a tiny Groq call with `max_tokens=3` that asks "is this a real JD? yes or no." Catches multi-word gibberish that passes the word count. If `False` → 422, analysis never happens.

ChromaDB lookup

```
chunks = query_resume(body.file_id, body.job_description)
if not chunks:
    raise HTTPException(status_code=404, detail="No resume data found for this file_id.")
```

Calls `embedder_service.query_resume()`. Converts the JD to a vector, does a cosine similarity search in ChromaDB filtered to this `file_id`, returns the top 5 most relevant resume text chunks.

GitHub context (optional)

```
github_context = None
if user:
    connection = get_github_connection(user["id"])
    if connection:
        github_context = await fetch_github_profile(connection["github_username"],
        connection["github_token"])
elif body.github_username and GITHUB_TOKEN:
    github_context = await fetch_github_profile(body.github_username, GITHUB_TOKEN)
```

Two paths: logged-in users use their OAuth-connected GitHub token; guests can manually type a username and the server uses its own PAT. `github_context` is either a dict of repo data or `None`. Passed to `analyze_resume` — if `None`, the GitHub section is omitted from the prompt entirely.

The main LLM call

```
result = analyze_resume(chunks, body.job_description, github_context)
result["github_enriched"] = github_context is not None
```

Calls `llm_service.analyze_resume()` — the actual Groq call. Returns `{ score, matched_skills, missing_skills, qualification_gaps, summary }` as a plain Python dict. Tags it with `github_enriched` so the frontend knows whether GitHub data factored in.

Save to history

```
if user:
    save_analysis_result(body.file_id, result["score"], ...)
```

Only for logged-in users. Calls `history_service.save_analysis_result()` → INSERTs a new row into Supabase's `resume_history` table. Guests skip this.

Return

```
return AnalyzeResponse(**result)
```

`**result` unpacks the dict into keyword arguments. FastAPI validates it against `AnalyzeResponse`, serializes to JSON, sends back with 200.

Full call chain

```
ResumeUpload.tsx
→ POST /api/resume/analyze { file_id, job_description }
  → get_current_user()      validates JWT → user dict or None
  → word count check        no external calls
  → is_valid_job_description() fast Groq yes/no
  → query_resume()          ChromaDB cosine similarity search
  → fetch_github_profile()   GitHub API (optional)
  → analyze_resume()         main Groq call → score + skills
  → save_analysis_result()   Supabase INSERT (logged-in only)
← AnalyzeResponse JSON
  → written into ResumeContext
  → score ring + skills rendered in UI
```

How Cosine Similarity Works in the Code

Cosine similarity is not written manually — it's configured once and then handled automatically by ChromaDB.

Configuration (`embedder_service.py`)

```
_collection = client.get_or_create_collection(
    name="resumes",
    metadata={"hsw:space": "cosine"},
)
```

"hsw:space": "cosine" tells ChromaDB to use cosine similarity when searching this collection. HNSW (Hierarchical Navigable Small World) is the indexing algorithm that makes search fast — it doesn't compare your query against every stored vector one by one, it navigates a graph structure to find nearest neighbours efficiently.

On upload — storing vectors

```
embeddings = _get_model().encode(chunks, show_progress_bar=False).tolist()
_get_collection().add(
    documents=chunks,
    embeddings=embeddings,
    ids=[f"{file_id}_{i}" for i in range(len(chunks))],
    metadatas=[{"file_id": file_id, "filename": filename, "chunk_index": i} ...]
)
```

Each resume chunk becomes a 384-number vector stored alongside the raw text and metadata. The `file_id` in metadata is how every query stays scoped to one user's resume.

On analyze — searching vectors

```
def query_resume(file_id: str, query: str, n_results: int = 5) -> list[str]:
    query_embedding = _get_model().encode([query]).tolist()
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=n_results,
        where={"file_id": file_id},
    )
    return results["documents"][0]
```

The JD text is embedded using the **same model** as the resume chunks. This is critical — if you used different models, the vectors would live in different mathematical spaces and comparison would be meaningless. ChromaDB computes cosine similarity between the JD vector and every stored chunk vector for that `file_id`, returns the 5 closest.

The full picture

On upload:

resume chunks → all-MiniLM-L6-v2 → 384-dim vectors → stored in ChromaDB

On analyze:

job description → all-MiniLM-L6-v2 → 384-dim vector

↓
ChromaDB cosine similarity search
(angle between JD vector and each chunk vector)
↓
top 5 most similar chunk texts returned
↓
passed to Groq for scoring

Why the same model matters: Cosine similarity measures the angle between two vectors. That angle only means something if both vectors were produced by the same model with the same understanding of language. Same model = same vector space = valid comparison.

End-to-End Flow 1: Resume Upload

```
User drops PDF/DOCX → clicks "Upload & Analyze"
|
▼ POST /api/resume/upload (multipart/form-data)
[resume_service.py]
1. Validate content type – PDF or DOCX only (400 if not)
2. Read all bytes into memory, enforce 5 MB limit (400 if over)
3. Generate UUID → upload bytes to Supabase Storage bucket "resumes"
   (no file ever written to disk)
|
▼
[parser_service.py]
4. Pass bytes as BytesIO to pdfplumber (PDF) or python-docx (DOCX) → plain text
5. Regex scan each line for section keywords → detected sections list
|
▼
[chunker_service.py]
6. Split text into 200-word overlapping windows (40-word overlap)
   Why overlap? A sentence crossing a boundary appears in both chunks
   so it's never missed during retrieval
|
▼
[embedder_service.py]
7. all-MiniLM-L6-v2 (local, ~90MB) encodes each chunk → 384-dim vector
8. Store in ChromaDB: { text, vector, file_id, chunk_index }
|
▼
Return { file_id, filename, word_count, sections } → frontend
```

Frontend saves the response into `ResumeContext` — a React Context mounted above the router. Navigating to Interview Prep and back doesn't lose the data because the context lives outside the page components.

An upload row is also INSERTed into `resume_history` (score=null) to anchor the `file_id` for history grouping.

End-to-End Flow 2: ATS Analysis

```

User pastes job description → clicks "Analyze Match"
    |
    ▼ POST /api/resume/analyze { file_id, job_description }
[resume.py route]
- < 20 words → 422 immediately (no LLM call)
- ≥ 20 words → llm_service.is_valid_job_description() – fast yes/no Groq call
- Not a real JD → 422 "This doesn't look like a real job description"
    |
    ▼
[embedder_service.query_resume()]
- Embed the JD using the same all-MiniLM-L6-v2 model
- ChromaDB cosine similarity search, filtered to this file_id
- Returns top-k most semantically similar resume chunks
    |
    ▼
[llm_service.analyze_resume(chunks, job_description)]
- Sends chunks + JD to Groq (llama-3.3-70b-versatile)
- Scores across TWO dimensions: skills/keywords AND stated qualifications
  (years of experience, degree requirements, certifications)
- Returns JSON: { score, matched_skills, missing_skills, qualification_gaps, summary
}
    |
    ▼
Return AnalyzeResponse → frontend shows:
- Animated score ring (color: green ≥70, amber ≥45, red <45)
- Matched skills (green badges)
- Missing skills (red badges)
- Qualification gaps (amber warning box – stated JD requirements not met)
- Plain-English summary
- "What does this mean?" link → score guide section
- "Prep for this interview" button → Interview Prep page

```

Why INSERT instead of UPDATE for history? Each analysis is a new row so users can score the same resume against multiple job descriptions and see all results in History. Updating in-place would overwrite the previous score.

Why RAG instead of pasting the whole resume? The JD is embedded into a vector and compared against resume chunk vectors. Only the *most relevant* parts of the resume go into the LLM prompt — lower cost, better focus, and it scales to long documents.

End-to-End Flow 3: Interview Prep

Two modes. One always works without a JD, one is personalised.

Behavioral Practice

Questions are **hardcoded on the frontend** — 15 questions across 5 categories (Self, Conflict, Leadership, Growth, Motivation). No API call, instant load. User expands a question, types an answer, requests AI feedback.

Role-Specific

```

User clicks "Prep for this interview" after ATS analysis
| (React Router passes { file_id, job_description } as state)
▼ POST /api/interview/questions { file_id, job_description }
[interview.py route]
- < 20 words → 422 immediately
- is_valid_job_description() → 422 if gibberish (error shown on submit, not while
typing)
|
▼
[embedder_service.query_resume(file_id, job_description)]
- Same ChromaDB lookup – pulls resume chunks most relevant to the JD
|
▼
[interview_service.generate_questions(chunks, job_description)]
- Sends chunks + JD to Groq
- Prompt requests 8 questions: 4 behavioral (role/culture tailored)
+ 4 technical (referencing specific tools/skills from JD)
- Returns JSON array of { question, category, hint }
|
▼
Frontend renders question cards (All view or One-by-one view)

```

Voice Answer Recording

```

User clicks mic button (Record) next to textarea
|
▼ MediaRecorder.getUserMedia({ audio: true })
- auto-picks audio/mp4 (Safari) or audio/webm;codecs=opus (Chrome)
- button turns red + pulsing while recording
|
User clicks Stop
|
▼ POST /api/interview/transcribe (multipart audio file)
[interview_service.transcribe_audio(bytes, filename, content_type)]
- Calls Groq whisper-large-v3
- Returns { text }
|
▼
Transcript appended to existing answer text in the textarea

```

AI Feedback on Answers

```

User types/records answer → clicks "Get AI Feedback"
|
▼ POST /api/interview/feedback { question, user_answer, file_id }
[embedder_service.query_resume(file_id, question)]
- Embeds the question to find relevant resume chunks
|
▼
[interview_service.get_feedback(question, answer, chunks)]

```

- Prompt to Groq is strict: if answer < 15 words or a non-answer ("don't know"), respond with one direct sentence – no hollow praise
- For real answers: evaluate STAR structure, specificity, measurable outcomes; reference specific resume experiences the candidate missed

|



Feedback displayed below the textarea

End-to-End Flow 4: Chatbot

Floating chat widget visible on all pages once authed. Auto-injects resume + JD context when available.

```
User opens chat panel → types message → hits Enter
|
▼ POST /api/chat/ { messages, file_id?, job_description? }
[chat.py route]
- if file_id present: query_resume(file_id, last_user_message) → top resume chunks
|
▼
[chat_service.stream_chat(messages, resume_chunks, job_description)]
- Builds system prompt:
  "You are HireReady's career AI..."
  + [if chunks] USER'S RESUME: {chunks}
  + [if jd] TARGET JOB DESCRIPTION: {jd[:1500]}
  + [if no resume] direct user to upload/select rather than paste in chat
- Calls Groq with stream=True
- Yields each token as it arrives
|
▼ StreamingResponse (text/event-stream)
data: {"token": "Here"}\n\n
data: {"token": " are"}\n\n
...
data: [DONE]\n\n
|
▼ Frontend ReadableStream
- Each token appended to the last assistant message in state
- Auto-scrolls to bottom
```

Key design: The chatbot has context only when a resume is loaded in the current session (uploaded or selected via "Use previous resume"). Clicking "Use previous resume" sets `parseResult.file_id` in `ResumeContext`, which the chatbot picks up automatically — previous resumes are readable, they just need to be selected first.

End-to-End Flow 6: Resume History

```
User navigates to /history
|
▼ GET /api/resume/history (Authorization: Bearer <JWT>)
[history_service.get_user_history(user_id)]
- Fetches all rows from resume_history for this user, ordered by uploaded_at asc
```

- Groups by `file_id` in Python:
 upload rows (`score=null`) → set filename + uploaded_at for the group
 analysis rows (`score!=null`) → append to `analyses[]` list
- Returns list sorted newest-first; analyses within each file sorted newest-first

|

▼

Frontend renders accordion:

- One card per `file_id` with filename, upload date, latest score badge
- Click to expand → all ATS analyses for that resume
- Each analysis: score badge, date, "View summary" toggle, skill chips, "Prep interview"
- Trash icon → inline confirmation → `DELETE /api/resume/history/{file_id}`
 |
 ▼ `DELETE /api/resume/history/{file_id}`
`[history_service.delete_resume_record(user_id, file_id)]`
 - Deletes all Supabase rows where `user_id` + `file_id` match
`[embedder_service.delete_chunks(file_id)]`
 - `collection.get(where={"file_id": file_id})` → finds all chunk IDs
 - `collection.delete(ids=...)` → removes vectors

Why delete ChromaDB chunks on history delete? The `file_id` is orphaned once the Supabase rows are gone — vectors would accumulate forever. `delete_chunks` is in `embedder_service.py` so the architectural rule holds: one service owns one storage concern.

End-to-End Flow 7: GitHub OAuth

User clicks "Connect GitHub" on home upload card

|

▼ GET `/api/github/connect` (Authorization: Bearer <JWT>)

[github_auth.py]

- Reads Origin header from request (e.g. `http://127.0.0.1:5173`)
- Validates against `FRONTEND_URLS` whitelist
- `payload = "{user_id}|{origin}"`
- `state = base64(payload) + "." + HMAC-SHA256(payload)[:24]`
- Redirects browser to GitHub authorize URL with state param

|

▼ User authorizes → GitHub redirects to:

GET `/api/github/callback?code=...&state=...`

[github_auth.py]

- Decodes base64, verifies HMAC signature
- Extracts `user_id` + origin from payload
- Exchanges code for GitHub access token
- Saves token to Supabase
- Redirects to `{origin}/home?github=connected`

Why encode origin in state? `localhost` and `127.0.0.1` are separate localStorage origins in browsers. If the callback hardcodes `localhost` but the user came from `127.0.0.1`, the Supabase session is invisible and they see a login page. Embedding the actual `Origin` header into the HMAC-signed state fixes this permanently.

Supabase Schema

```
-- resume_history table (one row per upload OR per analysis)
file_id      text      -- UUID from upload
user_id      uuid      -- foreign key to auth.users
filename     text      -- original filename
score        int       -- null for upload rows; 0-100 for analysis rows
matched_skills jsonb    -- array of strings
missing_skills jsonb    -- array of strings
jd_snippet   text      -- first 300 chars of the pasted JD
summary      text      -- Groq-generated plain-English summary
uploaded_at  timestampz
```

`get_user_history` groups these rows by `file_id` in Python — upload rows (`score=null`) anchor the card header, analysis rows become the nested `analyses` list. The frontend never sees blank cards.

Embeddings: Local vs Cloud

An embedding is a list of numbers representing the *meaning* of text. Two sentences with similar meaning → vectors that are close together, even if they share no words.

```
"5 years of Python experience"      → [0.12, -0.87, 0.34, ...]
"built Python microservices since 2019" → [0.13, -0.85, 0.31, ...] ← close!
```

	Local (sentence-transformers)	Cloud (OpenAI text-embedding-3-small)
Cost	Free	~\$0.00002/1K tokens
Dimensions	384	1536
Privacy	Data stays on server	Data leaves your machine
Setup	Download once (~90MB)	API key required
Swap effort	One line in <code>embedder_service.py</code>	One line in <code>embedder_service.py</code>

ChromaDB

Stores vectors so they can be searched later. Lives at `backend/chroma_db/` — SQLite-backed, persists across restarts.

Each chunk stored as:

- `id` : `{file_id}_{chunk_index}`
- `document` : raw text
- `embedding` : 384 floats
- `metadata` : `{ file_id, filename, chunk_index }`

The `where={"file_id": file_id}` filter in every query ensures you only search the current user's resume — not everyone's. Similarity is measured by **cosine similarity** (angle between vectors — smaller

angle = more similar meaning).

Groq vs Other LLM Options

Provider	Model	Cost	Key needed?
Groq	llama-3.3-70b	Free tier	Yes (free)
OpenAI	gpt-4o-mini	~\$0.001/req	Yes (paid)
Anthropic	claude-haiku-4-5	~\$0.001/req	Yes (paid)
Ollama	llama3 (local)	Free	No

Groq is used now because it's free and fast. Switching to OpenAI or Anthropic for production is a single import change in `llm_service.py` and `interview_service.py` — nothing else in the codebase is aware of which provider is running.

Note: `llama3-8b-8192` was decommissioned by Groq mid-project. Always check console.groq.com/docs/deprecations if you get a `400 model_decommissioned` error.

Authentication Flow

Supabase is a hosted backend-as-a-service built on PostgreSQL. It provides auth, a relational database, and file storage — all from one service. For auth, it runs its own Auth server alongside your database.

```
User submits email + password on AuthGate (route /)
|
▼ supabase.auth.signInWithPassword()
Supabase Auth server
- Checks credentials against internal auth.users table
- Returns a JWT (JSON Web Token) — a signed string encoding user ID, email, and expiry
- Supabase JS client stores JWT in localStorage automatically
|
▼
AuthContext.onAuthStateChanged fires
- Sets user + session in React state
- axios.defaults.headers.common['Authorization'] = `Bearer <token>`
- All future API calls now carry the token automatically
|
▼
React Router sees user is non-null → redirects from / to /home
|
▼ Any protected API call (e.g. POST /api/resume/upload)
FastAPI — get_current_user dependency (auth.py)
- Extracts Bearer token from Authorization header
- Calls supabase.auth.get_user(token) — Supabase verifies signature
- Returns { id, email } if valid; raises 401 if forged/expired
```

Why the JWT is safe to pass around: The token is cryptographically signed by Supabase using a secret key only they hold. Your backend never sees the user's password — only the signed token. If anyone

tampers with it, the signature check fails and it's rejected.

Session refresh: JWTs expire after 1 hour. The Supabase JS client silently refreshes them using a refresh token (also in localStorage). `onAuthStateChange` fires on each refresh and updates the axios header.

Guest mode: "Continue without signing in" stores a `guestMode` flag in `sessionStorage`. The app treats this as authenticated for routing purposes. Signing out clears the flag and returns to the auth gate.

Backend auth dependencies (auth.py):

- `get_current_user` — optional auth. Returns user dict or `None`. Existing endpoints stay backward-compatible.
- `require_user` — throws 401 if no valid token. Used when an endpoint must be locked to signed-in users.

Supabase Storage

Files are stored in Supabase Storage (a hosted S3-compatible object store) — no local disk involved. On upload, bytes are read into memory and uploaded directly to the `resumes` bucket with the UUID as the key. `parser_service.py` receives bytes and opens them via `BytesIO`, so `pdfplumber` and `python-docx` never touch disk.

```
Supabase Storage bucket: resumes/<uuid>.pdf (or .docx)
ChromaDB:                file_id → embedded chunks
resume_history table:    file_id + user_id → scores, metadata
```

The `file_id` is the key that ties all three together. Deleting a resume means: delete the Supabase history rows, delete the ChromaDB chunks. The storage object itself is retained (could be cleaned up with a Supabase storage delete call if needed).

The next step for production would be adding a proper `files` table with `user_id` as a foreign key, so the app can list a user's uploaded files without relying on `resume_history` upload rows.

Interview Talking Points

"Walk me through a user request end to end."

The user uploads a PDF. React sends it as multipart/form-data to `POST /api/resume/upload`. The service layer validates type and size, saves with a UUID filename, extracts text using `pdfplumber`, splits into 200-word overlapping chunks, embeds each chunk with a local sentence-transformers model, and stores the vectors in ChromaDB tagged with a `file_id`. When the user pastes a job description, we embed that too, query ChromaDB for the most semantically similar resume chunks, and send them to Groq to produce an ATS score and gap analysis. The same `file_id` and JD then flow to the interview prep page, where Groq generates role-specific questions grounded in both the resume and the JD.

"What is RAG and why does this project use it?"

RAG is Retrieval-Augmented Generation — instead of sending a full document to the LLM every time, you index it once as vectors and retrieve only the relevant parts at query time. Here, the resume is chunked and embedded on upload. At analysis time, the job description is embedded and the closest resume chunks are retrieved via cosine similarity. Only those chunks go into the Groq prompt. This keeps costs low, avoids context window limits, and lets the LLM focus on the most relevant content.

"Why local embeddings instead of OpenAI?"

Local embeddings with sentence-transformers cost nothing and keep user data on the server. The model downloads once (~90 MB) and runs in-process as a lazy singleton. Quality is very good for semantic similarity on short resume chunks. Switching to OpenAI's text-embedding-3-small in production is one line in `embedder_service.py` — nothing else in the codebase knows which model produced the vectors.

"How does state persist when the user navigates between pages?"

React Router unmounts page components on navigation, which normally destroys local state. We store the persistent data — parse result, ATS analysis, job description — in a React Context (`ResumeContext`) mounted above the router in the component tree. Context survives navigation because it's never unmounted. Ephemeral state like drag/drop and error messages stays local to the component since it doesn't need to persist.

"How does authentication work in this app?"

Authentication is handled by Supabase. On sign-in, Supabase's auth server verifies the credentials and returns a JWT. The Supabase JS client stores the JWT in `localStorage` and auto-refreshes it before expiry. Our `AuthContext` listens for auth state changes and sets the JWT as the default Authorization header on every axios request — so all API calls carry it automatically without any component needing to think about it. On the backend, a FastAPI dependency (`get_current_user` in `auth.py`) extracts the token and calls `supabase.auth.get_user()` to verify the signature. If the token is forged or expired, Supabase rejects it and we return a 401. The current endpoints support optional auth — they still work without a token — but `require_user` is available to lock down endpoints that must be tied to an account. When the user logs out, `ResumeProvider` is keyed on `user?.id ?? 'logged-out'` so it remounts with fresh empty state — the next login can't accidentally see the previous user's resume data.

"How does the resume history work, and why INSERT instead of UPDATE?"

History is stored in a Supabase table called `resume_history` . There are two row types: upload rows (score=null, created on file upload) and analysis rows (score set, inserted on each ATS run). We INSERT new rows instead of updating because a user might analyze the same resume against five different job descriptions — UPDATE would silently overwrite every previous score. The `get_user_history` service groups rows by `file_id` in Python so the frontend sees one accordion card per resume with a nested list of all scores. Deleting a resume removes all Supabase rows for that `file_id` and calls `delete_chunks()` in `embedder_service.py` to clean up the ChromaDB vectors so they don't accumulate as orphans.

"How does GitHub OAuth work and what was tricky about it?"

It's a standard three-leg OAuth flow — redirect to GitHub, get a code back, exchange for a token. The tricky part was the post-callback redirect. In development the frontend runs on both `localhost:5173` and `127.0.0.1:5173` , which browsers treat as completely separate `localStorage` origins. If you hardcode the redirect to `localhost` but the user came from `127.0.0.1` , the Supabase session is invisible after the redirect and they see a login page. The fix: read the `Origin` header from the `/connect` request, encode it with the user ID as `user_id|origin` , sign the payload with HMAC-SHA256 using the GitHub client secret, and pass it as the OAuth state parameter. The callback decodes and verifies it, then redirects to whichever origin is in the payload — so `localhost` and `127.0.0.1` both work correctly without any hardcoded URLs.

"How does the chatbot work and what context does it have?"

The chatbot is a floating widget that streams responses token-by-token using Server-Sent Events. When the user sends a message, the frontend POSTs to `/api/chat/` with the conversation history plus the current `file_id` and `job_description` from React Context. The backend queries ChromaDB with the last user message to retrieve the most relevant resume chunks, then builds a system prompt with those chunks and the JD. That prompt goes to Groq with `stream=True` — each token is yielded as a `data: {"token": "..."} SSE event`. The frontend reads the stream with a `ReadableStream` reader and appends each token to the last message in state, giving a real-time typing effect. If no resume is loaded, the system prompt instructs the LLM to direct the user to upload or select one — it never asks users to paste their resume in the chat window.

"How does voice recording work across Safari and Chrome?"

The Web Speech API only works in Chrome/Edge, so we use `MediaRecorder` + Groq Whisper instead, which works in all modern browsers. The `VoiceMicButton` component calls `getUserMedia({ audio: true })` for mic permission, then starts a `MediaRecorder`. It detects the best supported MIME type — `audio/webm;codecs=opus` for Chrome, `audio/mp4` for Safari — using `MediaRecorder.isTypeSupported()`. On stop, it assembles the recorded chunks into a `Blob` and POSTs it as multipart form data to `/api/interview/transcribe`. The backend strips the codec suffix from the content-type header, calls Groq's `whisper-large-v3` model, and returns the transcript. The transcript is appended to whatever the user has already typed, so typing and speaking can be mixed freely.

"How do you prevent garbage input from getting an ATS score or interview questions?"

Two layers. First, a word count check — fewer than 20 words gets rejected immediately before any LLM call. Second, a semantic check: a fast Groq call with `max_tokens=3` asks "is this a real job description?" and returns yes/no. This costs almost nothing but catches multi-word gibberish that passes the word count. Both checks live in shared code — `is_valid_job_description()` in `llm_service.py` — so `/analyze` and `/questions` use the exact same validation without duplication. On the frontend, the error only appears when the user clicks submit, not while they're typing — this avoids distracting mid-composition warnings.

"How would you scale this for real users?"

Files move from local disk to Supabase Storage (one line in `resume_service.py`). ChromaDB moves to Pinecone or Weaviate (one line in `embedder_service.py`). A Supabase PostgreSQL table maps file IDs to user accounts — the `file_id` already acts as the key across the entire pipeline, so adding a `user_id` foreign key is the main schema change. Groq swaps to OpenAI or Anthropic for production-grade SLAs (one line in the LLM services). Supabase's Row Level Security lets us enforce "users can only see their own rows" at the database level without any backend code change.

"Why FastAPI over Flask or Django?"

FastAPI has async support out of the box — critical for LLM API calls that take several seconds. Pydantic handles request validation automatically (wrong shape → 422, no manual checks). Auto-generated OpenAPI docs at `/docs`. Django is overkill for an API-first backend; Flask requires wiring Pydantic and async manually.

"What is CORS and why do you need it?"

CORS is a browser policy that blocks requests to a different origin than the page was loaded from. Frontend is on port 5173, backend on 8000 — different origins. FastAPI's `CORSMiddleware` adds `Access-Control-Allow-Origin` headers so the browser permits the cross-origin requests.

What's Built

Feature	Where
Resume upload + Supabase Storage	POST /api/resume/upload, resume_service.py
PDF/DOCX parsing via BytesIO (no disk writes)	parser_service.py
Text chunking (overlapping windows)	chunker_service.py
Local embeddings + ChromaDB storage	embedder_service.py
ATS scoring via Groq (skills + qualifications)	POST /api/resume/analyze, llm_service.py
Qualification gaps warning (amber UI)	AnalyzeResponse.qualification_gaps, ResumeUpload.tsx
Semantic JD validation (gibberish detection)	llm_service.is_valid_job_description(), used in /analyze + /questions
Matched/missing skills + score ring	ResumeUpload.tsx
Resume history (accordion, nested analyses)	GET /api/resume/history, history_service.py, HistoryPage.tsx
Delete full resume from history	DELETE /api/resume/history/{file_id}, delete_chunks()
Delete individual ATS score from history	DELETE /api/resume/history/{file_id}/score?at=, delete_analysis_entry()
ATS summary saved + shown in history	summary col in resume_history, HistoryPage.tsx
Context-aware upload card header	App.tsx HomePage reads parseResult/analyzeResult
Logo click resets state (with confirm modal)	AppInner.handleLogoClick(), App.tsx
State reset on login/logout	<ResumeProvider key={user?.id ?? 'logged-out'}>
GitHub OAuth (origin-aware HMAC state)	github_auth.py
JD URL fetch (direct HTTP + Jina fallback)	POST /api/resume/fetch-jd, jd_fetcher_service.py
Role-specific interview questions	POST /api/interview/questions, interview_service.py
Behavioral question bank (15 questions)	InterviewPage.tsx (hardcoded)
AI feedback on answers	POST /api/interview/feedback, interview_service.py

Voice answer recording → Whisper transcription	POST /api/interview/transcribe, VoiceMicButton in InterviewPage.tsx
All / One-by-one question views	InterviewPage.tsx
Chatbot (SSE streaming, RAG context)	POST /api/chat/, chat_service.py, ChatBot.tsx
Cross-page state persistence	ResumeContext.tsx
Nav + SVG logo + user avatar	App.tsx, Logo.tsx
ATS score guide section	App.tsx
Auth gate (sign in / sign up / guest)	AuthGate.tsx, route /
Supabase JWT auth — frontend	AuthContext.tsx, lib/supabase.ts
Supabase JWT auth — backend	core/auth.py, core/supabase.py
Profile page (account info + sign out)	ProfilePage.tsx, route /profile

What's Next

Feature	Notes
PostgreSQL	Proper files table with user_id FK; move off resume_history rows
MCP integrations	LinkedIn JD fetch, deeper GitHub profile integration

Architectural Tradeoffs

These are the decisions worth being able to explain and defend.

1. ChromaDB (local) vs Cloud Vector DB (Pinecone, Weaviate)

What it is: ChromaDB runs on the same machine as FastAPI. It's SQLite-backed — just files on disk at `backend/chroma_db/` . No network calls, no accounts needed.

	ChromaDB (local)	Pinecone/Weaviate (cloud)
Cost	Free	\$70+/month at scale
Latency	~5ms (same machine)	~50–200ms (network hop)
Setup	<code>pip install chromadb</code>	API keys, account, config
Persistence	Survives server restarts	Survives server restarts + crashes
Serverless compatible	No	Yes
Scale	Single server only	Horizontally scalable

The real risk — serverless deployments (e.g. Vercel):

A normal server is always running with persistent disk access. A serverless platform like Vercel spins up a fresh container per request and throws it away after. There is no persistent filesystem. So:

```
Request 1 → Container A → embeddings written to /tmp/chroma_db ✓
Request 2 → Container B (fresh) → /tmp/chroma_db doesn't exist ✗
```

This has nothing to do with logging out. Logging out is just a browser action — it clears a JWT. The problem is that between any two requests, Vercel might give you a completely new container. ChromaDB assumes a persistent filesystem; serverless doesn't provide one.

For production: Replace ChromaDB with Pinecone or Supabase's pgvector extension. Only `embedder_service.py` changes — nothing else in the codebase knows which vector store is running.

2. Supabase Storage (cloud) vs Local Disk for Resume Files

What it is: Raw PDF/DOCX bytes go directly to Supabase's `resumes` bucket on upload. The server never writes a file to its own disk.

	Supabase Storage	Local Disk
Survives server restart	Yes	No
Multi-server deployments	Yes (shared)	No (each server has its own copy)
Cost	Generous free tier	Free (your disk)
Complexity	Needs bucket config	<code>open(path, 'wb')</code>
Privacy	Third party stores the files	Files stay on your server

Why BytesIO matters: `parser_service.py` takes the raw bytes and opens them with `BytesIO` — an in-memory file-like object. Neither `pdfplumber` nor `python-docx` ever see a real file path. The bytes come in over HTTP, go to Supabase and to the parser simultaneously in memory, and nothing hits disk. This makes the backend stateless — restart it, and nothing is lost because no state ever lived on its disk.

3. Stateless JWT vs Server-Side Sessions

What it is: JWTs are self-contained signed tokens. The server verifies them cryptographically without looking up anything in a database.

	JWT (stateless)	Server sessions
Server memory needed	Zero	Grows with active users
Instant revocation	Hard (token lives until expiry)	Easy (delete session row)
Horizontal scaling	Trivial (any server can verify)	Needs shared session store (Redis)
Complexity	Supabase handles it entirely	You manage session storage

The tradeoff most people miss: You can't instantly log someone out with JWTs. If a token has a 1-hour expiry and you want to forcibly revoke it, the token is still valid until it expires — you can't "un-sign" it. Supabase mitigates this with short expiry (1 hour) + silent background refresh using a separate refresh token. For this app it's a non-issue.

Why stateless matters for scaling: With a normal session, every server needs to be able to look up "is session ID abc123 valid?" — which means either one central database (bottleneck) or a shared cache like Redis. With JWTs, any server that has the signing key can verify any token independently. No coordination needed.

4. Groq vs OpenAI

	Groq	OpenAI
Speed	Very fast (custom LPU hardware)	Moderate
Cost	Free tier, very cheap	More expensive
Model quality	Good (Llama 3.3 70B)	GPT-4o is stronger
Reliability	Smaller company, newer	Industry standard SLA
Voice (Whisper)	Same model, much cheaper	Same model, pricier

Groq's LPU (Language Processing Unit): Custom silicon designed specifically for LLM inference — not repurposed GPU hardware. Makes streaming feel noticeably faster. For a demo, the difference is visible.

Why this decision is low-risk: All Groq calls are isolated to `llm_service.py` and `interview_service.py`. Switching to OpenAI is changing two import lines and two API call formats — nothing in routes, context, or the frontend knows which LLM is running.

5. Local Embeddings vs API Embeddings

	Local (sentence-transformers)	OpenAI text-embedding-3-small
Cost	Free after one-time download	~\$0.00002/1K tokens
Privacy	Resume text never leaves server	Sent to OpenAI
Cold start	~2–3s first request (model loads)	Instant
Vector size	384 dimensions	1536 dimensions
Quality	Good for resume matching	Slightly better

Why privacy matters here: Resumes contain personal information — names, addresses, employment history. With local embeddings, that text only goes to Groq (for scoring) and Supabase (for storage, explicitly consented to). It never goes to a third-party embeddings API. That's a selling point for enterprise use.

The cold start: The `all-MiniLM-L6-v2` model is loaded as a lazy singleton — it loads on the first embedding request after startup and stays in memory. First request after a cold start takes 2–3 seconds. Every subsequent request is fast.

6. RAG vs Sending the Whole Resume

What it is: RAG (Retrieval-Augmented Generation) = index once as vectors, retrieve only relevant parts at query time, send only those to the LLM.

Alternative: Paste the entire resume text into every LLM prompt.

	RAG (this app)	Full document in every prompt
Cost per request	Low (only relevant chunks)	High (full resume every time)
Context window limit	Not a concern	Breaks for long resumes
Focus	LLM sees only relevant content	LLM must process everything
Upfront work	Must chunk + embed on upload	Nothing on upload
Latency	Extra ChromaDB lookup	Simpler, slightly faster

For this use case RAG is the right call: A resume is 500–1500 words. The JD is another 500–1000. Together they approach or exceed context limits for some models. More importantly, RAG makes the LLM focus — when scoring a "Python engineering" JD, you want the LLM to see your Python projects, not your education section.

Demo Script

Lead with what the app does for the user. Explain the architecture only when asked.

Opening (30 seconds)

"Most people apply to jobs without knowing if their resume will get past the ATS filter — the automated system that screens resumes before a human ever sees them. This app tells you your score, what's missing, and then helps you prep for the interview."

Step 1 — Upload

Drag the resume in. While it processes:

"The backend is parsing the PDF in memory using pdfplumber, splitting the text into overlapping chunks, and embedding each chunk with a local ML model. Those embeddings go into ChromaDB — a local vector database. This is what lets the chatbot and interview prep semantically search your resume later without re-reading the whole document every time."

Step 2 — Analyze

Paste a JD, click Analyze. While it processes:

"It embeds the job description into a vector, finds the most semantically similar resume chunks via cosine similarity, and sends those chunks to Groq's Llama 3.3 70B model to produce a structured score."

When results appear:

"You get a score, matched skills, missing skills — and qualification gaps, which are things the JD explicitly requires that your resume doesn't address. Like '5 years required' when your resume doesn't state years."

Step 3 — Interview Prep

Click "Prep for this interview":

"Because the app already has my resume and the JD in context, questions start generating immediately. These aren't generic — Groq sees both my resume and the role requirements and generates questions

grounded in what I actually wrote."

Show feedback:

"The feedback prompt explicitly bans hollow praise. If your answer is weak, it tells you why and what to add."

Step 4 — Chatbot

Open the chat widget:

"The chatbot has my resume as context. It uses the same RAG pipeline — my message gets embedded, ChromaDB returns the most relevant resume chunks, and those go into the system prompt. I can ask 'do I have any DevOps experience?' and it's searching my actual resume to answer."

Step 5 — History

Navigate to History:

"Every ATS analysis is saved to Supabase — a cloud Postgres database. I can come back days later, see all my scores across different job descriptions, and delete individual entries. Deleting a resume removes both the database rows and the ChromaDB vectors so they don't accumulate as orphans."

Common follow-up questions and tight answers

"Why Groq instead of OpenAI?"

"Groq runs on custom LPU hardware — significantly faster and cheaper than GPU-based providers. And because all LLM calls are isolated to two service files, switching providers is a one-afternoon job."

"How does the chatbot know my resume?"

"RAG pipeline. Message gets embedded, ChromaDB finds the most relevant resume chunks via cosine similarity, chunks go into the system prompt. The LLM never re-reads the whole resume — just the relevant parts."

"Could this scale?"

"Two things to swap: ChromaDB for Pinecone or pgvector — needed because ChromaDB requires a persistent filesystem and serverless platforms don't provide one. And a proper files table in Postgres with user_id as a foreign key. Everything else — Supabase, Groq — is already cloud-native. Both swaps are single-file changes by design."

"How does auth work?"

"Supabase issues a JWT on sign-in. The JS client stores it and auto-refreshes it before expiry. Every API call sends it in the Authorization header. The FastAPI backend verifies the signature with Supabase on each request — fully stateless, no session storage server-side."

"Why local embeddings instead of OpenAI?"

"Two reasons: cost and privacy. Resumes contain personal data — with local embeddings it never leaves the server to go to a third-party API. And it's free. Quality is sufficient for semantic similarity on short resume chunks. Swapping to OpenAI embeddings in production is one line in `embedder_service.py`."

"What is a vector embedding?"

"A list of numbers that encodes the meaning of text. Two sentences with similar meaning produce vectors that are mathematically close together, even if they share no words. 'Five years of Python experience' and 'built Python microservices since 2019' produce vectors with a small angle between them. That's what cosine similarity measures — the angle."

"Why INSERT instead of UPDATE for history rows?"

"A user might analyze the same resume against five different job descriptions. UPDATE would silently overwrite the previous score. INSERT gives you a full history. The tradeoff is the table grows, but each row is tiny — it's the right call for this use case."